2026-05-05-p1-f13-lsp-integration-design

Phase 1 / Feature 13 — LSP Integration

Date: 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Bring real Language-Server Protocol diagnostics into HelixCode so the agent (and the user) get IDE-grade error/warning feedback without running a full build. Five language servers ship in the curated allowlist:

Binary	Extensions	Source
`gopls`	`.go`	`go install golang.org/x/tools/gopls@latest` (or distro)
`rust-analyzer`	`.rs`	`rustup component add rust-analyzer`
`pyright`	`.py`	`npm i -g pyright`
`typescript-language-server`	`.ts`, `.tsx`, `.jsx`, `.js`	`npm i -g typescript-language-server typescript`
`clangd`	`.c`, `.cpp`, `.cc`, `.h`, `.hpp`	`apt install clangd` / `brew install llvm`

Servers are NOT bundled with HelixCode — F13 detects them at runtime via `exec.LookPath` and lazy-spawns on first relevant file open. Unknown extensions are a no-op (no LSP, no error). Idle timeout is 5 minutes per server, then SIGTERM + reaper.

Three concrete user surfaces ship together:

1. **Agent tools** — `LSPGetDiagnostics` and `LSPAnalyzeDiagnostic` registered with `internal/tools/registry.go::ToolRegistry`. The agent can ask for diagnostics whenever it wants.
2. **Auto-trigger after Edit/Write** — every successful `fs_edit`, `fs_write`, and `multi_edit_commit` tool call triggers a diagnostic refresh for the touched file; the resulting `DiagnosticSummary` is **inlined into the tool result** so the agent sees diagnostics on the next turn without an extra round-trip.
3. **CLI surface** — `/lsp` slash command (`status` / `restart [name]` / `list-servers` / `stop [name]`) and `helixcode lsp` cobra subcommand with the same verbs.

JSON-RPC transport and LSP protocol types are NOT hand-rolled. F13 adopts two new external dependencies — `go.lsp.dev/jsonrpc2` and `go.lsp.dev/protocol` — which are the

canonical Go LSP libraries (used by `gopls` itself for its tests). Hand-rolling a JSON-RPC 2.0 framing layer + LSP type registry is exactly the class of work where bluffs hide; using the upstream library is the conservative call.

The scope of F13 is **diagnostics only**. Code completion, go-to-definition, hover, find-references, rename, workspace symbols, inlay hints, and semantic tokens are explicitly deferred (§8).

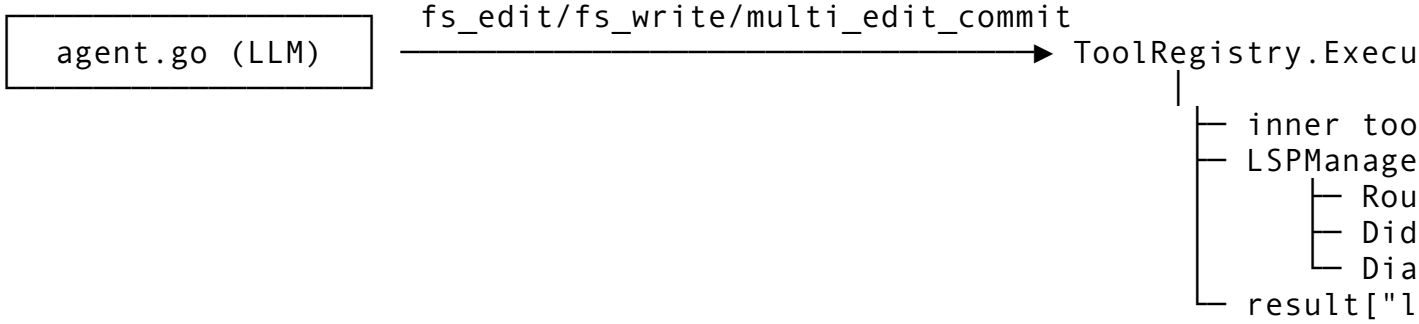
2. Architecture

Three layers, each with a single responsibility, all under `internal/tools/`:

- `LSPManager` — owns server lifetimes. Lazy-spawns a server on first relevant file, registers a 5-minute idle timer per server, routes file-extension lookups to the right `LSPClient`, restarts on crash with exponential backoff, and shuts down all servers on `Close()`.
- `LSPClient` — one per running language server. Wraps a `*jsonrpc2.Conn` over the server's stdin/stdout, performs the `initialize` / `initialized` / `shutdown` / `exit` handshake, tracks open documents, dispatches `textDocument/didOpen` / `didChange` / `didClose`, and consumes `textDocument/publishDiagnostics` notifications into an in-memory map keyed by `DocumentURI`.
- **Agent tools** — `LSPGetDiagnosticsTool` and `LSPAnalyzeDiagnosticTool` query the `LSPManager` and return `DiagnosticSummary` shaped for the agent.

The auto-trigger lives in `ToolRegistry.Execute` (the existing chokepoint where every tool call funnels through; see registry.go:334). After a successful `fs_edit` / `fs_write` / `multi_edit_commit`, the registry calls `LSPManager.NotifyChange(ctx, absPath)` which:

1. Dispatches `textDocument/didChange` on the matching client (or `didOpen` if first time).
2. Waits up to 800ms for fresh diagnostics (cancellable via ctx).
3. Returns a `DiagnosticSummary` that the registry attaches to the tool result map under key `lsp_diagnostics` (only when non-empty — empty diagnostics stay silent, no noise).



`LSPManager` also implements the `/lsp` and `helixcode lsp` surface verbs as plain Go method calls (no separate IPC).

3. Components

3.1 New files

- `helix_code/internal/tools/lsp_types.go` — `Diagnostic`, `DiagnosticSummary`, `DiagnosticSeverity`, `LSPServerSpec`, `ServerStatus`. No deps.
- `helix_code/internal/tools/lsp_client.go` — `LSPClient` (jsonrpc2 wrapper, handshake, didOpen/didChange, publishDiagnostics handler).
- `helix_code/internal/tools/lsp_client_test.go` — unit tests against an in-process fake server (in-memory `io.ReadWriter` pair).
- `helix_code/internal/tools/lsp_manager.go` — `LSPManager` (lazy spawn, idle timeout, file-extension router, crash recovery).
- `helix_code/internal/tools/lsp_manager_test.go` — unit tests with the in-tree fake LSP subprocess (built once via `go test -c`).
- `helix_code/internal/tools/lsp_servers.go` — curated allowlist (`gopls`, `rust-analyzer`, `pyright`, `typescript-language-server`, `clangd`), `Detect()` via `exec.LookPath`.
- `helix_code/internal/tools/lsp_servers_test.go`.
- `helix_code/internal/tools/lsp.go` — `LSPGetDiagnosticsTool`, `LSPAnalyzeDiagnosticTool` (Tool interface adapters).
- `helix_code/internal/tools/lsp_test.go`.
- `helix_code/internal/tools/lsp_fakeserver/main.go` — small in-tree LSP server that implements `initialize`, `textDocument/didOpen`, `textDocument/didChange`, and emits a deterministic `publishDiagnostics` (used by manager tests + Challenge harness).
- `helix_code/internal/commands/lsp_command.go` — `/lsp` slash command with `status` / `restart` / `list-servers` / `stop` subcommands.
- `helix_code/internal/commands/lsp_command_test.go`.
- `helix_code/cmd/cli/lsp_cmd.go` — `helixcode lsp` cobra root + 4 subcommands.
- `helix_code/cmd/cli/lsp_cmd_test.go`.
- `helix_code/tests/integration/lsp_test.go` — `//go:build integration`, gated per §5.
- `helix_code/tests/integration/cmd/p1f13_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f13-lsp-integration/CHALLENGE.md` + `run.sh`.

3.2 Modified files

- `helix_code/internal/tools/registry.go` — register `LSPGetDiagnosticsTool` + `LSPAnalyzeDiagnosticTool` in `registerAllTools()`; add `SetLSPManager(*LSPManager)` and a post-Execute hook that calls `lspManager.NotifyChange(ctx, path)` when the tool name is `fs_edit` / `fs_write` / `multi_edit_commit` and the inner Execute returned no error. Augment the tool result map with `"lsp_diagnostics"` when the summary is non-empty. Add a `CategoryLSP ToolCategory = "lsp"` constant.
- `helix_code/cmd/cli/main.go` — construct an `LSPManager` during CLI startup, wire it into the registry via `SetLSPManager`, register `/lsp` slash command and `helixcode lsp` cobra subcommand, defer `lspMgr.Close()`.
- `helix_code/go.mod` / `helix_code/go.sum` — add `go.lsp.dev/jsonrpc2 v0.10.0` and `go.lsp.dev/protocol v0.12.0`. (Both confirmed absent from

go.mod at spec time; only `aws-sdk-go-v2/.../protocol/eventstream` is present, which is unrelated.)

3.3 Types

```
// internal/tools/lsp_types.go

type DiagnosticSeverity int
const (
    SeverityError       DiagnosticSeverity = 1
    SeverityWarning     DiagnosticSeverity = 2
    SeverityInformation DiagnosticSeverity = 3
    SeverityHint        DiagnosticSeverity = 4
)

type Diagnostic struct {
    ID       string
        `json:"id"`        // stable hash for
        LSPAnalyzeDiagnostic lookup
    FilePath string             `json:"file_path"` // absolute
        path on host
    Range    DiagnosticRange    `json:"range"`
    Severity DiagnosticSeverity `json:"severity"`
    Code     string             `json:"code,omitempty"`
    Source   string             `json:"source"`    // "gopls",
        "rust-analyzer", etc.
    Message  string             `json:"message"`
}

type DiagnosticRange struct {
    Start Position `json:"start"`
    End   Position `json:"end"`
}
type Position struct {
    Line      int `json:"line"`
    Character int `json:"character"`
}

type DiagnosticSummary struct {
    TotalErrors      int           `json:"total_errors"`
    TotalWarnings    int           `json:"total_warnings"`
    TotalInformation int           `json:"total_information"`
    TotalHints       int           `json:"total_hints"`
    Diagnostics      []Diagnostic  `json:"diagnostics"`     //
        truncated to 5 when Expandable
    Expandable       bool          `json:"expandable"`      //
        true if more than 5
    Source           string        `json:"source,omitempty"`//
        e.g. "auto-trigger:fs_edit"
```

```
}

type LSPServerSpec struct {
    Name       string   // "gopls"
    Binary     string   // "gopls"
    Extensions []string // [".go"]
    Args       []string // []
}

type ServerStatus struct {
    Name       string
    Spec       LSPServerSpec
    Running    bool
    PID        int
    StartedAt  time.Time
    LastActive time.Time
    OpenDocs   int
}

// internal/tools/lsp_client.go

type LSPClient struct {
    spec        LSPServerSpec
    cmd         *exec.Cmd
    conn        *jsonrpc2.Conn       // go.lsp.dev/jsonrpc2
    rootURI     protocol.DocumentURI // go.lsp.dev/protocol
    diagnostics map[protocol.DocumentURI][]Diagnostic
    openDocs    map[protocol.DocumentURI]int32 // version counter
    mu          sync.Mutex
}

func NewLSPClient(ctx context.Context, spec LSPServerSpec, root
        string) (*LSPClient, error)
func (c *LSPClient) DidOpen(ctx context.Context, path string,
        content []byte) error
func (c *LSPClient) DidChange(ctx context.Context, path string,
        content []byte) error
func (c *LSPClient) DidClose(ctx context.Context, path string)
        error
func (c *LSPClient) GetDiagnostics(path string) []Diagnostic
func (c *LSPClient) WaitForDiagnostics(ctx context.Context, path
        string, timeout time.Duration) []Diagnostic
func (c *LSPClient) Status() ServerStatus
func (c *LSPClient) Close(ctx context.Context) error

// internal/tools/lsp_manager.go

type LSPManagerOptions struct {
```

```
    Root        string                          // workspace root
        (project dir)
    IdleTimeout time.Duration                   // default
        5*time.Minute
    Specs       []LSPServerSpec                 // curated allowlist
    Detect      func(string) (string, error) // default
        exec.LookPath; injectable for tests
    Logger      *log.Logger                     // optional
}

type LSPManager struct {
    opts    LSPManagerOptions
    clients map[string]*LSPClient // by spec.Name
    timers  map[string]*time.Timer
    mu      sync.Mutex
    closed  bool
}

func NewLSPManager(opts LSPManagerOptions) *LSPManager
func (m *LSPManager) NotifyChange(ctx context.Context, absPath
        string) (*DiagnosticSummary, error)
func (m *LSPManager) GetDiagnostics(ctx context.Context, absPath
        string, minSeverity DiagnosticSeverity)
        (*DiagnosticSummary, error)
func (m *LSPManager) Status() []ServerStatus
func (m *LSPManager) Restart(ctx context.Context, name string)
        error
func (m *LSPManager) Stop(ctx context.Context, name string) error
func (m *LSPManager) Close(ctx context.Context) error
```

`Diagnostic` is HelixCode's own type, NOT the `protocol.Diagnostic` from go.lsp.dev. We deliberately wrap because (a) we add the stable `ID` field (needed for `LSPAnalyzeDiagnostic` lookup), (b) we add `FilePath` on every record (the protocol type only has it on the publish notification), and (c) we want a stable schema for the JSON sent to the LLM that doesn't churn if the upstream library bumps. The wrap layer is ~30 LoC of straight field copies in `lsp_client.go`.

3.4 User surfaces

Agent tool calls (registered with `ToolRegistry`):

Tool name	Schema	Returns
`LSPGetDiagnostics`	`{file_path?: string, severity?: "error|warning|information|hint"}`	`DiagnosticSummary`
`LSPAnalyzeDiagnostic`	`{diagnostic_id: string}` (required)	`{diagnostic: Diagnostic, file_context:`

Tool name	Schema	Returns
		`string, suggestion?: string}`

Slash command `/lsp`: - `/lsp status` — table of all servers (name, running, PID, open docs, last active). - `/lsp list-servers` — show curated allowlist + which ones were detected on PATH. - `/lsp restart [name]` — restart one server (`gopls`) or all if no arg. - `/lsp stop [name]` — graceful shutdown of one server or all.

Cobra `helixcode lsp` — same four verbs:

```
helixcode lsp status
helixcode lsp list-servers
helixcode lsp restart [name]
helixcode lsp stop [name]
```

4. Data flow

4.1 Startup wiring (`cmd/cli/main.go`)

```
NewCLI()
  ├─ root := determineWorkspaceRoot()      // git toplevel or cwd (reuse F
  ├─ specs := tools.DefaultLSPServerSpecs()
  ├─ lspMgr := tools.NewLSPManager(LSPManagerOptions{
  │      Root: root, Specs: specs, IdleTimeout: 5*time.Minute,
  │  })
  ├─ registry.SetLSPManager(lspMgr)
  ├─ register `/lsp` slash + `helixcode lsp` cobra
  └─ defer lspMgr.Close(ctx)
```

4.2 Lazy spawn flow

```
LSPManager.NotifyChange(ctx, absPath)
  ├─ ext := filepath.Ext(absPath)
  ├─ spec := specByExt(ext)          // nil → no-op, return empty summary
  ├─ client := m.clients[spec.Name]
  ├─ if client == nil:
  │     bin, err := m.opts.Detect(spec.Binary)
  │     if err != nil:                // server not installed
  │         m.logOnce("LSP %s not on PATH — skipping diagnostics", spec.Na
  │         return empty, nil         // NOT an error
  │     client = NewLSPClient(ctx, spec, m.opts.Root)
  │     m.clients[spec.Name] = client
  ├─ resetIdleTimer(spec.Name)
  ├─ content := os.ReadFile(absPath)
  ├─ if client.IsOpen(absPath): client.DidChange(...) else: client.DidOpen
  ├─ diags := client.WaitForDiagnostics(ctx, absPath, 800*time.Millisecond
  └─ return summarize(diags, source="auto-trigger:<toolName>"), nil
```

4.3 Auto-trigger flow (in `registry.go::Execute`)

```
result, execErr := tool.Execute(ctx, params)
if execErr == nil && r.lspManager != nil && isEditTool(name):
    path := extractEditedPath(name, params)
    if path != "":
        summary, _ := r.lspManager.NotifyChange(ctx, path)  // never block
        if summary != nil && len(summary.Diagnostics) > 0:
            wrapResultWithLSP(result, summary)              // adds "lsp_d
```

`isEditTool` returns true for `fs_edit`, `fs_write`, `multi_edit_commit`. `extractEditedPath` reads `params["file_path"]` (Edit/Write) or, for multi-edit-commit, falls back to the first path in the staged batch. Errors in `NotifyChange` are logged at WARN; they NEVER fail the user's edit (CONST-035 requires a clear separation: the edit happened either way).

4.4 Idle timeout

On every successful `LSPClient` request, `LSPManager` resets a `time.AfterFunc` with `IdleTimeout` (default 5 min). On fire, the manager calls `client.Close(ctx)` (graceful: send `shutdown` request, then `exit` notification, then wait up to 2s, then SIGTERM, then SIGKILL after another 2s) and removes it from the map. Next `NotifyChange` for that language re-spawns.

4.5 Crash recovery

`LSPClient` watches `cmd.Wait()` in a goroutine. On unexpected exit, it marks itself dead. The manager attempts up to 3 restarts with exponential backoff (1s, 2s, 4s) before giving up. After permanent failure, the spec is marked `Disabled` until `/lsp restart <name>` is called manually.

4.6 `/lsp status` flow

Slash handler calls `LSPManager.Status() []ServerStatus`, renders a `text/tabwriter` table, returns it as a slash-command response.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Server binary not on PATH** — log once at INFO ("LSP gopls not on PATH; install via `go install golang.org/x/tools/gopls@latest`"), `NotifyChange` returns empty summary + nil err. The user's edit is never blocked.
- **Server fails to start** — log at WARN with the exit code + first 200 chars of stderr, mark spec `Disabled`, return empty summary + nil err.
- **Server crashes mid-session** — auto-restart with backoff (§4.5).
- `initialize` **handshake hangs** — 10s ctx timeout on `NewLSPClient`; on timeout, kill subprocess, return error to manager which falls back to "no diagnostics".
- **Unknown file extension** — `specByExt` returns nil, `NotifyChange` returns empty summary + nil err. Not an error.
- `textDocument/publishDiagnostics` **arrives for an unknown URI** — store under that URI key anyway; `WaitForDiagnostics` only consults the requested URI.

- **Concurrent `NotifyChange` for the same client** — guarded by `LSPClient.mu`; serialized to avoid out-of-order `didChange` versions.
- **Manager `Close` while a `NotifyChange` is in flight** — the call observes `closed=true` after acquiring the lock and returns empty summary.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F13 is "LSP test passed" with no real LSP server running. Most CI/dev machines do NOT have all five servers installed. Therefore the gating policy below is non-negotiable:

1. **Unit tests** (mocks OK ONLY at the jsonrpc2 transport boundary). `LSPClient` accepts a generic `io.ReadWriter` so unit tests can pair two `bytes.Buffer`-backed in-memory pipes and validate request shaping (`initialize` with the right `RootURI`, `didChange` with monotonically increasing version) and response parsing. NO mocks of the manager, NO mocks of `os/exec`.

2. **Integration tests** (`-tags=integration`) — gated on real binary presence:

   - `TestLSP_GoplsRoundTrip` — when `exec.LookPath("gopls") == nil`, runs a real `gopls` subprocess against a Go file with a deliberate `package` mismatch, asserts diagnostic comes back with `severity=error`. When `gopls` is absent: `t.Skip("SKIP-OK: P1-F13 gopls not installed (install: go install golang.org/x/tools/gopls@latest)")`.
   - `TestLSP_RustAnalyzerRoundTrip` — `rust-analyzer` absent → `t.Skip("SKIP-OK: P1-F13 rust-analyzer not installed (install: rustup component add rust-analyzer)")`.
   - `TestLSP_PyrightRoundTrip` — `pyright` absent → `t.Skip("SKIP-OK: P1-F13 pyright not installed (install: npm i -g pyright)")`.
   - `TestLSP_TypescriptLanguageServerRoundTrip` — absent → `t.Skip("SKIP-OK: P1-F13 typescript-language-server not installed (install: npm i -g typescript-language-server typescript)")`.
   - `TestLSP_ClangdRoundTrip` — absent → `t.Skip("SKIP-OK: P1-F13 clangd not installed (install: apt install clangd OR brew install llvm)")`.
   - `SKIP-OK:` is the canonical marker required by `make no-silent-skips`. Bare `t.Skip()` is forbidden.

3. **Challenge harness** — exercises the **manager pipeline** (lazy spawn, idle timeout, file-extension routing, crash recovery, auto-trigger) via the **in-tree fake LSP server** at `internal/tools/lsp_fakeserver/main.go`. The harness builds the fake server once with `go build -o /tmp/p1f13_fake ./internal/tools/lsp_fakeserver`, registers a synthetic `LSPServerSpec{Binary: "/tmp/p1f13_fake", Extensions: [".fake"]}` on the manager, writes a `.fake` file, observes the diagnostic round-trip, then waits past the idle timeout and asserts the subprocess is reaped. This proves the entire manager + client + auto-trigger pipeline works end-to-end with a real subprocess speaking real JSON-RPC over real stdio — it does NOT depend on any external language server.

4. **Anti-bluff rule in the Challenge:** the harness output MUST present TWO sections — `=== MANAGER PIPELINE (always runs, in-tree fake server) ===` and `=== REAL LANGUAGE SERVERS (gated) ===`. The second section

enumerates the 5 servers and prints `[PASS]` or `[skipped: <binary> not on PATH]` for each. The Challenge MUST NOT print `[PASS]` for a server that wasn't actually invoked. A reader of the Challenge output can never confuse "manager works" with "every language server works".

5. **Concrete forbidden phrases** (anti-bluff smoke clean check, applied to every new file):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO
        implement\|placeholder" \
  internal/tools/lsp.go internal/tools/lsp_client.go
        internal/tools/lsp_manager.go \
  internal/tools/lsp_servers.go internal/tools/lsp_types.go
        internal/tools/lsp_fakeserver \
  internal/commands/lsp_command.go cmd/cli/lsp_cmd.go &&
        echo BLUFF || echo clean
```

   Must always print `clean`.

6. **The fake server is NOT a mock of a language server.** It is a real OS subprocess that speaks real LSP-framed JSON-RPC 2.0 over stdin/stdout. It validates the same code paths a real `gopls` would exercise (initialize handshake, didOpen, publishDiagnostics, shutdown, exit, idle reap). It does not pretend to do semantic analysis — it emits a deterministic diagnostic for any opened document, which is enough to prove the pipeline.

6. Testing

6.1 Unit (mocks OK, transport boundary only)

- `TestLSPClient_InitializeHandshake_SendsRootURI` — paired in-memory pipes; assert outbound JSON contains expected `rootUri`.
- `TestLSPClient_DidOpen_FollowedBy_DidChange_VersionIncrement`.
- `TestLSPClient_PublishDiagnostics_StoredByURI`.
- `TestLSPClient_Shutdown_GracefulExit`.
- `TestLSPManager_LazySpawn_OnFirstRelevantFile`.
- `TestLSPManager_RouterByFileExtension`.
- `TestLSPManager_UnknownExtension_ReturnsEmptySummary_NoError`.
- `TestLSPManager_BinaryNotOnPath_ReturnsEmptySummary_NoError`.
- `TestLSPManager_IdleTimeout_ReapsServer` (uses fake-clock injection).
- `TestLSPManager_CrashRecovery_RestartsWithBackoff` (kills the fake subprocess mid-test).
- `TestLSPManager_Status_ReportsAllRunningServers`.
- `TestLSPManager_Close_ShutdownAllClients`.
- `TestLSPGetDiagnosticsTool_FiltersBySeverity`.
- `TestLSPAnalyzeDiagnosticTool_LookupByID`.
- `TestRegistry_AutoTriggerAfterFSEdit_AttachesLSPDiagnostics`.
- `TestRegistry_AutoTriggerSkipsOnEditError`.
- `TestRegistry_AutoTriggerNeverBlocksUserEdit_OnLSPError`.
- `TestLSPCommand_StatusRendersTable`.
- `TestLSPCommand_RestartByName`.
- `TestLSPCobra_StatusInvokesManager`.

6.2 Integration (`-tags=integration`, gated per §5.2)

- `TestLSP_GoplsRoundTrip` — real `gopls`; `SKIP-OK: P1-F13 gopls not installed (install: go install golang.org/x/tools/gopls@latest)`.
- `TestLSP_RustAnalyzerRoundTrip` — `SKIP-OK: P1-F13 rust-analyzer not installed (install: rustup component add rust-analyzer)`.
- `TestLSP_PyrightRoundTrip` — `SKIP-OK: P1-F13 pyright not installed (install: npm i -g pyright)`.
- `TestLSP_TypescriptLanguageServerRoundTrip` — `SKIP-OK: P1-F13 typescript-language-server not installed (install: npm i -g typescript-language-server typescript)`.
- `TestLSP_ClangdRoundTrip` — `SKIP-OK: P1-F13 clangd not installed (install: apt install clangd OR brew install llvm)`.
- `TestLSP_AutoTriggerThroughFSEdit_GoplsEndToEnd` (gated on `gopls` present) — exercises the full registry → manager → real gopls path.

6.3 Challenge

- **Phase 1 — manager pipeline (always runs)** — fake LSP server spawned, `.fake` file written, diagnostic round-trip observed, idle timeout reaps subprocess, crash + restart verified. ~10 [PASS] lines.
- **Phase 2 — real servers (gated)** — for each of the 5 servers: detect, attempt round-trip, print `[PASS]` or `[skipped: <reason>]`. Skipped lines name the missing binary AND the install command.
- Final summary line: `MANAGER=10/10 PASS; REAL_SERVERS=<n>/5 PASS (<5-n> skipped)`.

7. Cross-platform

Pure Go + stdio subprocess. The implementation depends on: - `os/exec.CommandContext` — same semantics on Linux, macOS, Windows. `cmd.StdinPipe()` / `cmd.StdoutPipe()` work identically across all three. (This is documented Go behaviour; tested extensively by `gopls` itself.) - `go.lsp.dev/jsonrpc2` and `go.lsp.dev/protocol` — pure Go, no cgo, no platform-specific code. - Process termination — graceful `shutdown`/`exit` LSP messages first; fall back to `cmd.Process.Signal(syscall.SIGTERM)` (which is mapped to `TerminateProcess` on Windows by the Go runtime); final fallback `cmd.Process.Kill()`. - Path handling — diagnostics carry absolute host paths; we use `filepath.Abs` and `filepath.ToSlash` for the LSP URI conversion (LSP requires forward slashes in `file://` URIs even on Windows).

No new platform-specific code required.

8. Out of scope (deferred)

- **Code completion** (`textDocument/completion`) — F13 is diagnostics-only.
- **Go-to-definition / find-references / rename / hover** — deferred to a future F13.5.
- **Workspace symbols** (`workspace/symbol`) — deferred.
- **Inlay hints / semantic tokens** — deferred.
- **Code actions / quick fixes** — `LSPAnalyzeDiagnostic` returns suggestion text only, NOT auto-applied edits.
- **Multi-root workspaces** — single workspace root per `LSPManager` instance in v1.

- **Custom server allowlist via config** — five-server allowlist is hardcoded in `lsp_servers.go`. User-defined extra servers are deferred.
- **Server-installed-on-demand** — F13 NEVER auto-installs language servers; install instructions are surfaced in skip messages instead.

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has TWO sections (manager pipeline vs real servers), the first always runs against a real subprocess speaking real JSON-RPC, the second is explicitly gated and never claims PASS without a runtime call. `[skipped: …]` lines name the missing binary so a reader can audit.
- **CONST-039** — F13 ships with a Challenge in `challenges/p1-f13-lsp-integration/` and an evidence harness at `tests/integration/cmd/p1f13_challenge/main.go`.
- **CONST-042 (No-Secret-Leak)** — LSP traffic over stdio frequently contains absolute paths to user files; `Diagnostic.Message` may contain user code snippets. Default INFO logging in `LSPClient` MUST NOT print full diagnostic messages or absolute paths; instead it logs `len(message)` and the basename only. DEBUG-level logging may include full content but is opt-in via `HELIX_LSP_DEBUG=1`. The Challenge verifies `INFO` log lines do not contain substrings of test diagnostic messages.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force.
- **No-Mocks-In-Production (Universal Rule 2)** — `LSPManager`, `LSPClient`, and the agent tools are real, talking to real subprocesses over real stdio. Mocks live only in `_test.go` files at the transport boundary and in the in-tree fake server (which is itself NOT a mock — it is a real subprocess used by both unit tests and the Challenge harness).

10. Open questions resolved

Q	Answer	Resolution
Q1: language server scope	(B) curated allowlist	gopls / rust-analyzer / pyright / typescript-language-server / clangd; routed by file extension
Q2: JSON-RPC + LSP types implementation	(B) external libraries	`go.lsp.dev/jsonrpc2 v0.10.0` + `go.lsp.dev/protocol v0.12.0` (both new deps)
Q3: server lifecycle	(B) lazy + idle timeout	spawn on first relevant file open, 5-minute idle timeout per server, managed by `LSPManager`
Q4: auto-trigger	(A) auto after Edit/Write	`ToolRegistry.Execute` post-hook for `fs_edit`/`fs_write`/`multi_edit_commit`; summary inlined into tool result under `lsp_diagnostics`
Q5: user surface	(A) full surface	`/lsp` slash + `helixcode lsp` cobra, both with

Q	Answer	Resolution
		`status` / `restart [name]` / `list-servers` / `stop [name]`